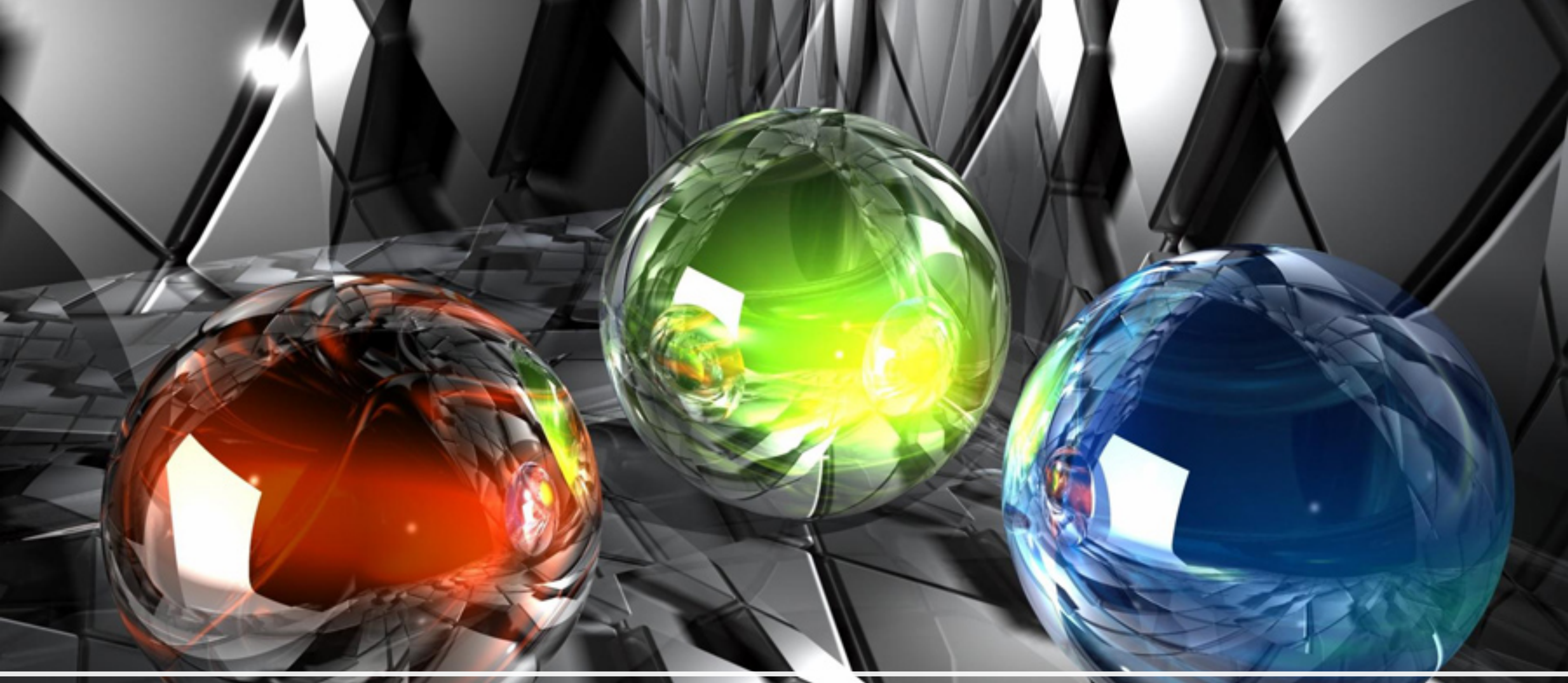


Лекция 14



Алгоритмы рендеринга 12-21

Лучевое литье (рейкастинг)

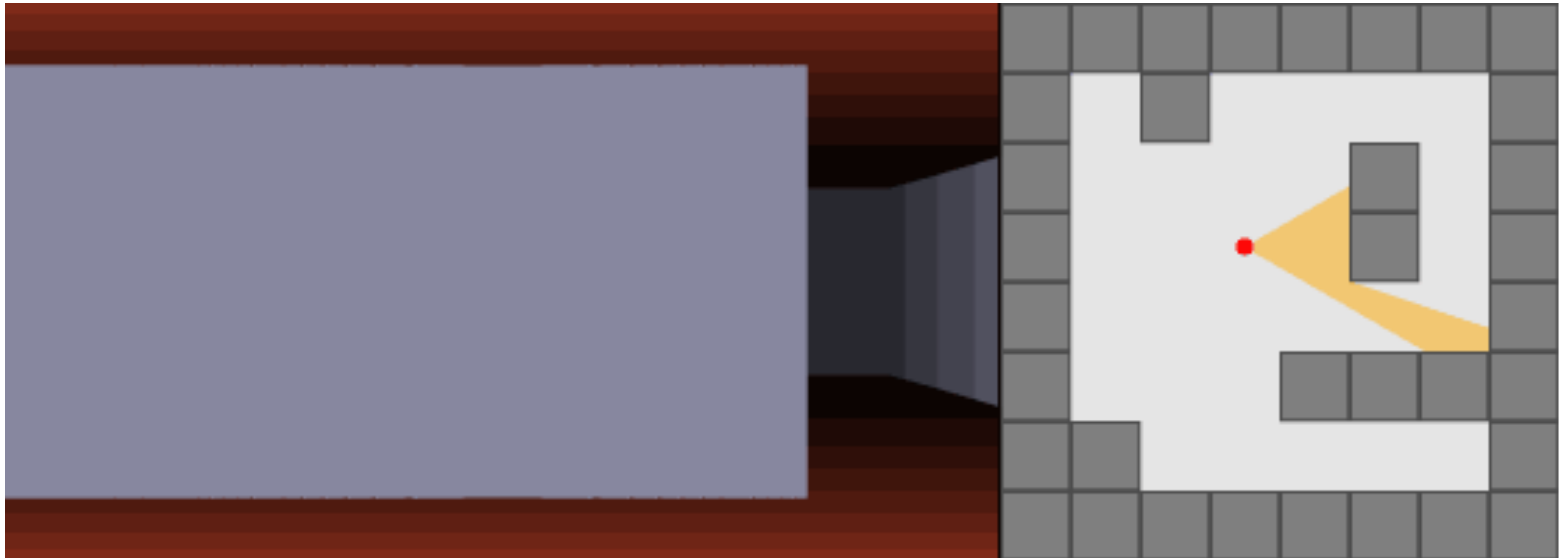
- «Рейкастинг (ray casting, „испускание лучей“) — самый простой из множества алгоритмов рендеринга компьютерной графики, использующих геометрический алгоритм трассировки лучей (ray tracing). Алгоритмы рендеринга на основе трассировки лучей работают на уровне изображений, выполняя рендеринг трёхмерных сцен в двухмерные изображения.
- Трассировка лучей (Ray Tracing) — это технология рендеринга изображений, которая позволяет достичь высокого уровня реалистичности благодаря имитации физики поведения света и теней.

Лучевое литье (рейкастинг)

- Принцип рейкастинга заключается в том, чтобы *попиксельно отслеживать испускаемые из глаза лучи, чтобы найти ближайший объект, препятствующий пути каждого луча* — представьте, что изображение — это проволочная сетка, а каждый квадрат сетки — это пиксель. Таким образом мы можем определить объект, который видит глаз через этот пиксель.
- Рейкастинг — это популярная фундаментальная техника, используемая для определения видимости определённых объектов (полигонов) трассировкой лучей из глаза (например, персонажа игрока) до каждого пикселя и нахождением самых ближайших пересечений с объектами.

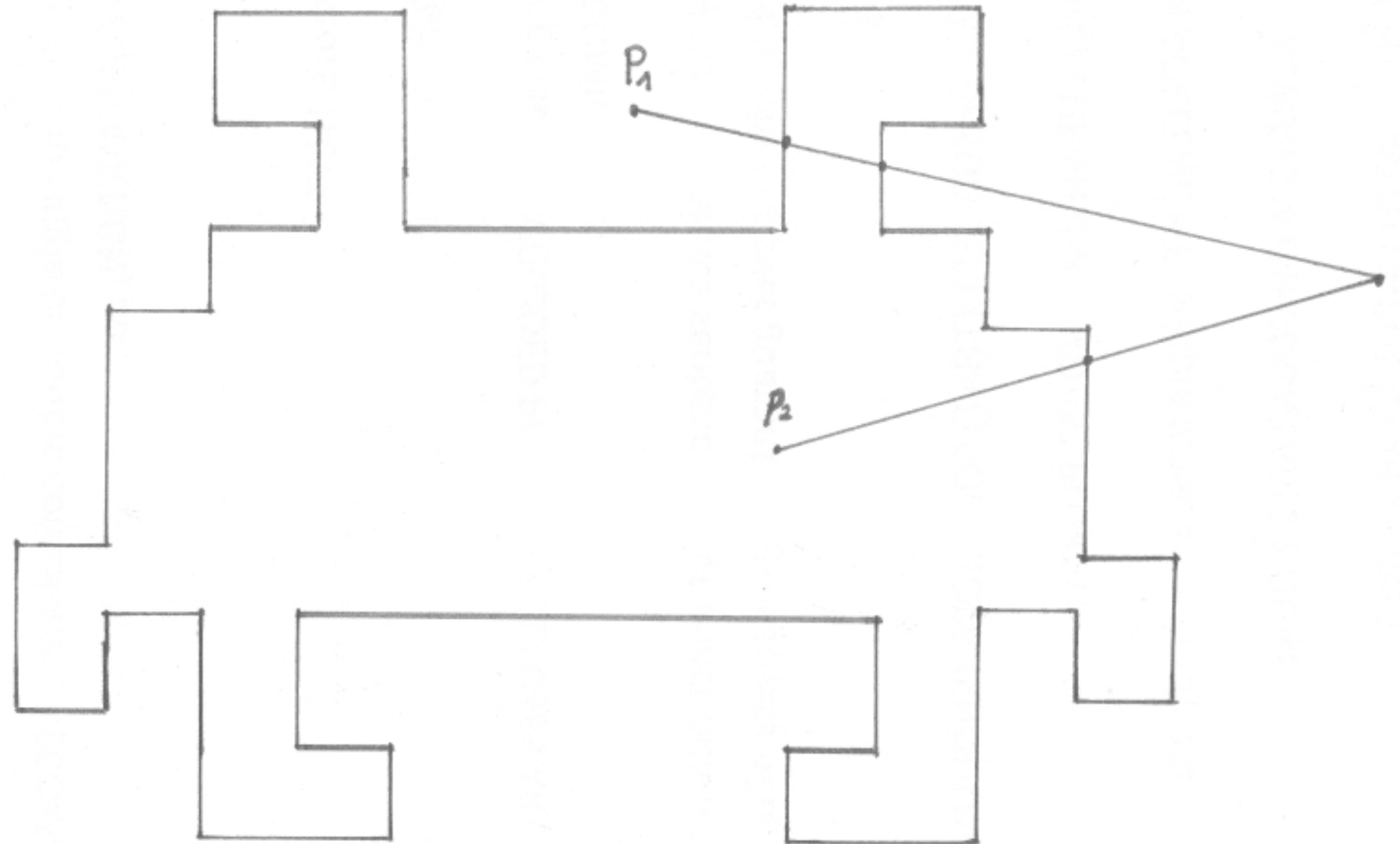
Где может использоваться рейкастинг? Создание 3D-перспективы на 2D-карте

Самой известной игрой, использующей эту технику, является Wolfenstein 3D. Лучи в ней трассировались для определения ближайших объектов, а их расстояние от позиции игрока использовалось для правильного масштабирования.



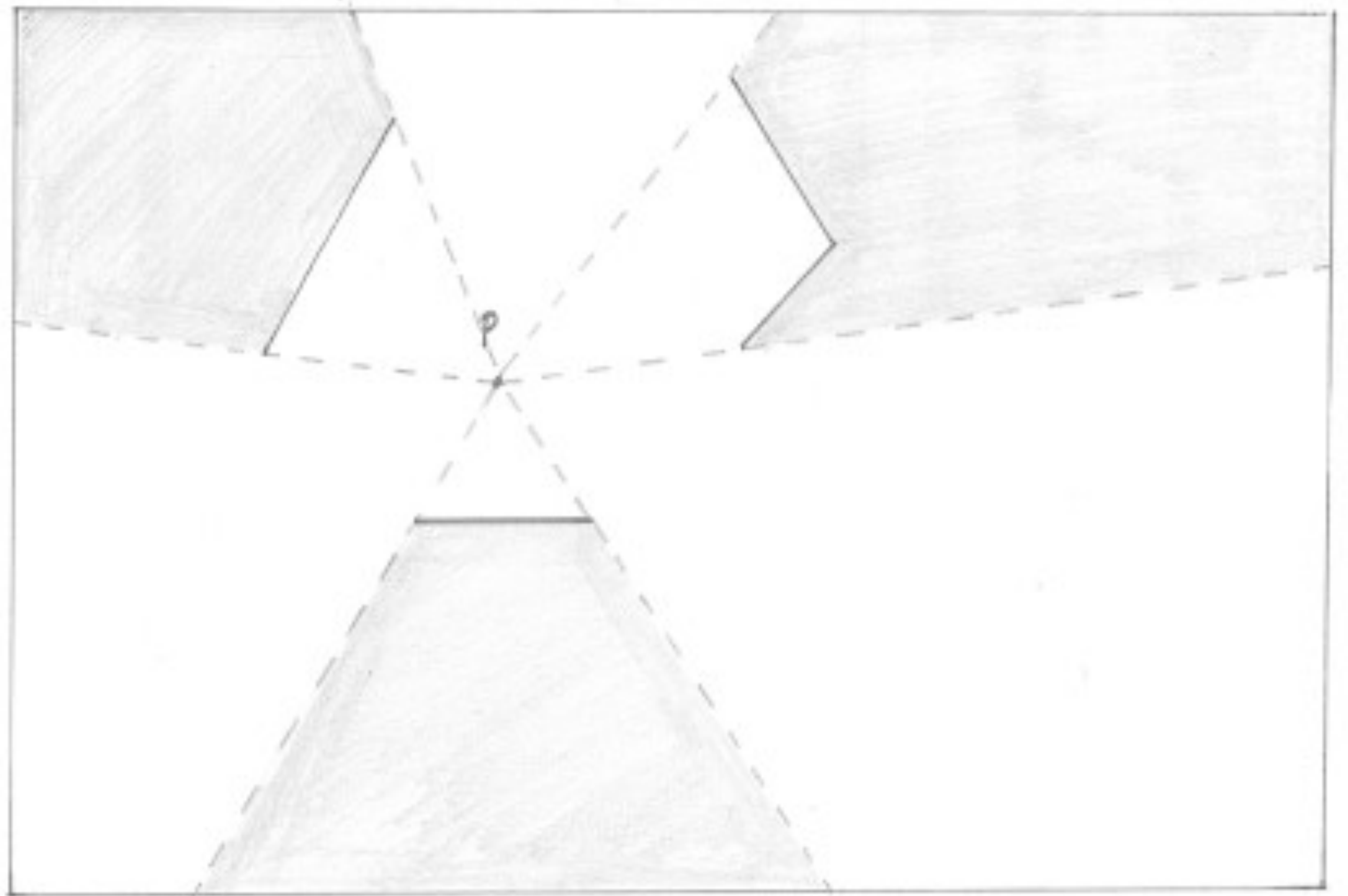
Где может использоваться рейкастинг? Задача точки в многоугольнике

- Задача PIP (point-in-polygon) заключается в определении того, находится ли точка внутри, снаружи или на границе многоугольника.
- При помощи алгоритма рейкастинга мы можем посчитать, сколько раз точка пересекает края многоугольника.
- Если количество пересечений чётно, то точка находится снаружи многоугольника.
- Если количество пересечений нечётно, то точка находится внутри или на границе многоугольника.



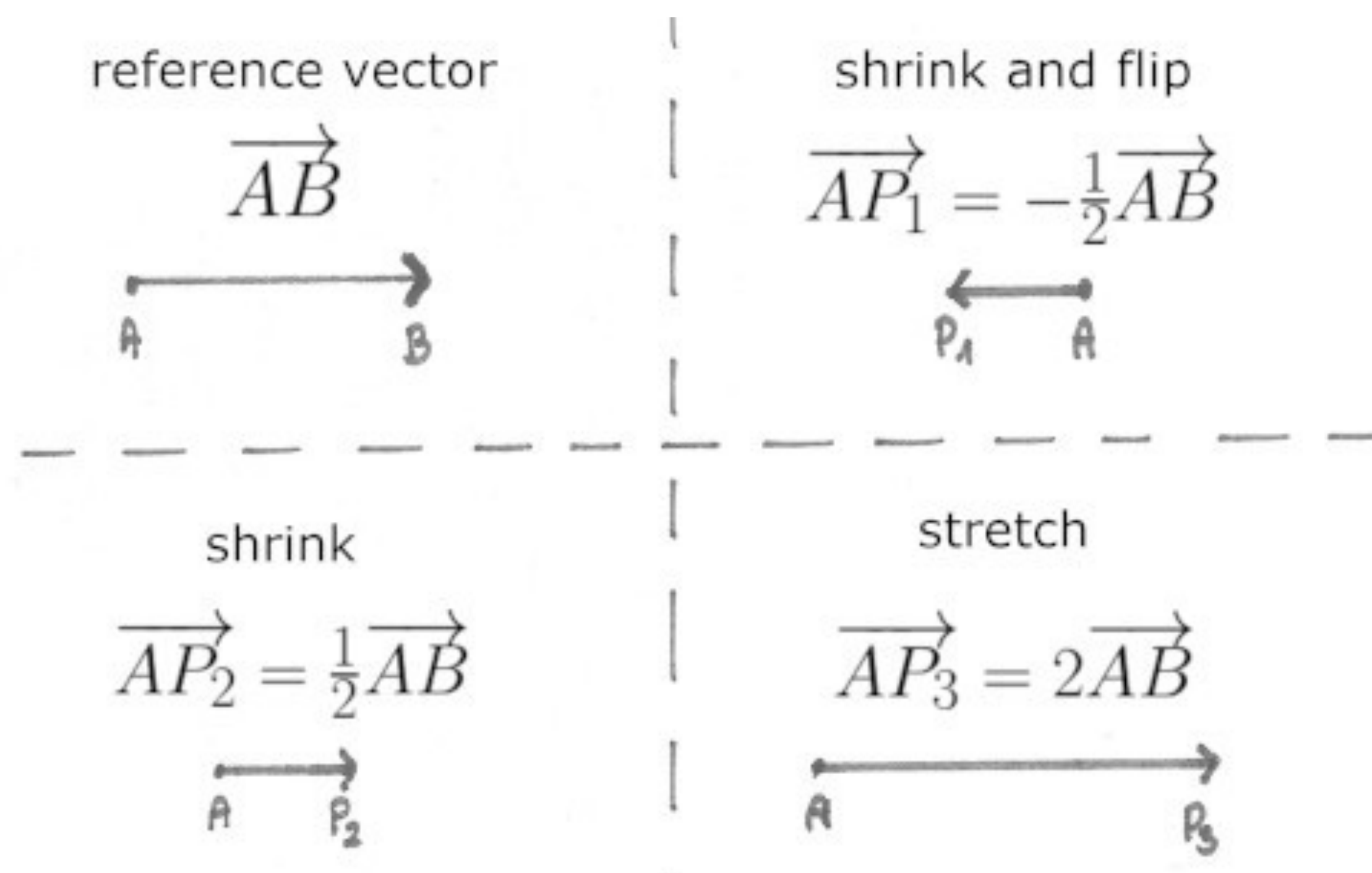
Где может использоваться рейкастинг? Видимость объектов и испускание света

Определение
видимости объектов
для игрока и освещение
видимой области.



Видимость объектов и испускание света в 2D-играх. Точка пересечения отрезка и луча. Вывод параметрического уравнения прямой

Мы можем выразить вектор $\overrightarrow{AP}$ следующим уравнением: $\overrightarrow{AP} = t\overrightarrow{AB}$, где t — параметр уравнения, определяющий, насколько мы растягиваем ($|t| > 1$) или сжимаем ($|t| < 1$), и меняем ли мы направление ($t < 0$) вектора $\overrightarrow{AP}$ относительно вектора $\overrightarrow{AB}$.



Вывод параметрического уравнения прямой

Как вы могли заметить, мы можем использовать t как коэффициент масштабирования:
 $|AP| = |t||AB|$. Мы воспользуемся этим при определении ближайшей точки пересечения.

Теперь мы можем легко увидеть, нужно ли нам, чтобы точка находилась на:

- прямой: $t \in \mathbb{R}$,
- луче: $t \geq 0$,
- отрезке прямой: $0 \leq t \leq 1$.

Вывод параметрического уравнения прямой

Выяснив всё это, мы наконец можем вывести параметрическое уравнение:

$$\overrightarrow{AP} = t\overrightarrow{AB}$$

$$P - A = t(B - A)$$

$$P = t(B - A) + A$$

Матрицы. Операции над матрицами

Проще говоря, матрица в 3D-графике — это массив чисел, расположенных по строкам и столбцам:

$$\begin{bmatrix} 0 & 4 & 8 & 12 \\ 1 & 5 & 9 & 13 \\ 2 & 6 & 10 & 14 \\ 3 & 7 & 11 & 15 \end{bmatrix}$$

$$\begin{bmatrix} \mathbf{m}_{00} & \mathbf{m}_{01} & \mathbf{m}_{02} & \mathbf{m}_{03} \\ \mathbf{m}_{10} & \mathbf{m}_{11} & \mathbf{m}_{12} & \mathbf{m}_{13} \\ \mathbf{m}_{20} & \mathbf{m}_{21} & \mathbf{m}_{22} & \mathbf{m}_{23} \\ \mathbf{m}_{30} & \mathbf{m}_{31} & \mathbf{m}_{32} & \mathbf{m}_{33} \end{bmatrix}$$

Матрицы. Операции над матрицами

- В 3D-приложениях используются матрицы либо размером 3×3 (матрицы вращения), либо 4×4 (матрица трансформации). Рассмотрим матрицу 4×4 (трансформацию).
- В первых трех столбцах находится вращение (*Rotation*), которое представлено в виде трех ортогональных друг к другу векторов (ось вправо, ось вперед, ось вверх), а по диагонали масштабирование (*Scale*).
- В четвертом столбце — смещение (*Translation*). А единица под смещением — это однородная координата. Если она равна 1 — позволяет сдвигать вектор перемножением матриц. А если равна 0 — *Translation* считается вектором направления.
- **Однородные координаты — способ представления точек и трансформаций в расширенном координатном пространстве.**

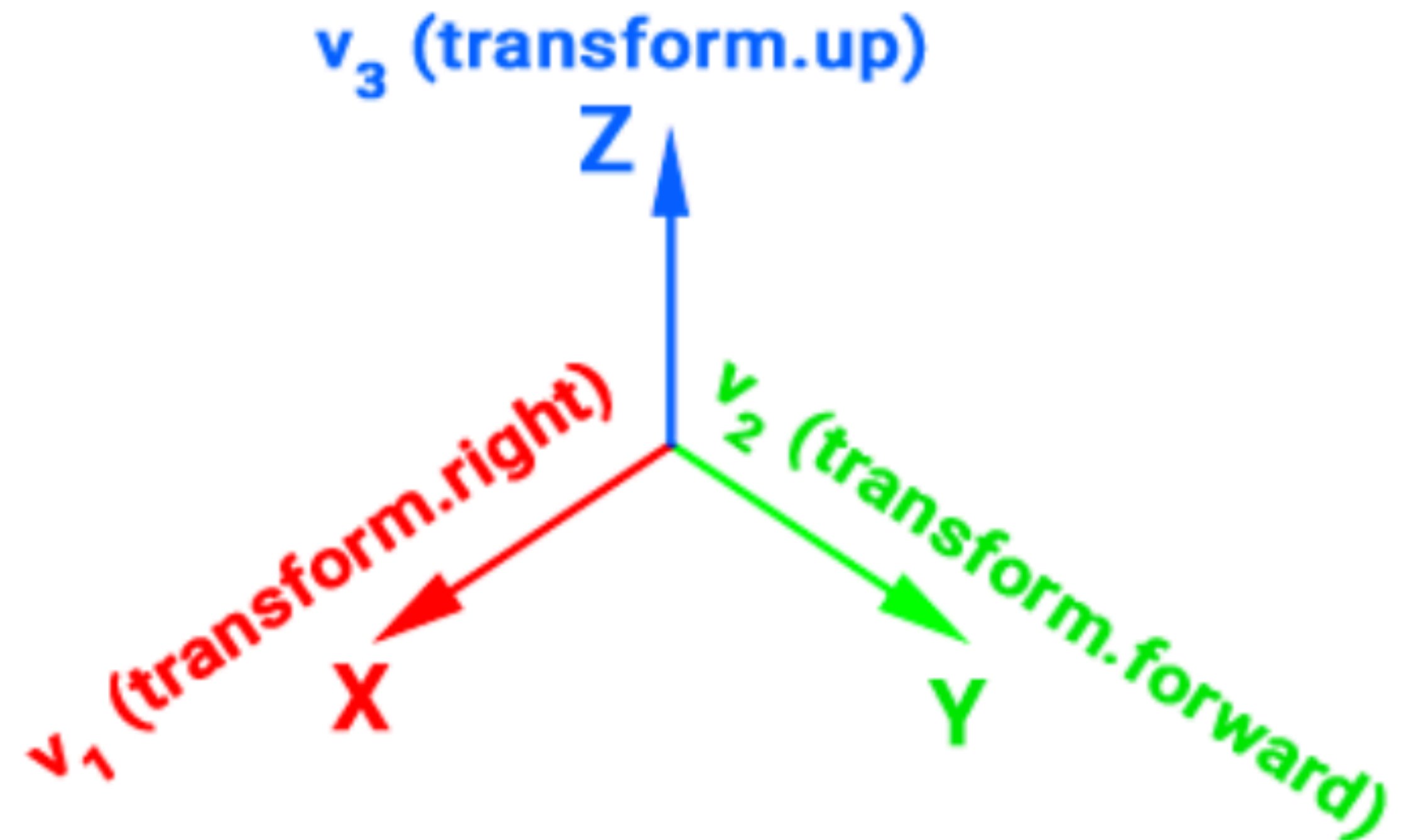
Матрицы. Операции над матрицами

- Как правило, для удобства работы все объекты в виртуальном мире формируются в определенные иерархии. Такие иерархии называются родителско-дочерними связями.
- *Например: пистолет в руке главного героя. «Пистолет» является дочерним к объекту «герой».*

rotation and scale

$$\begin{bmatrix}
 \begin{matrix} \text{v1.x} \\ S_x R_{00} \end{matrix} & \begin{matrix} \text{v2.x} \\ R_{01} \end{matrix} & \begin{matrix} \text{v3.x} \\ R_{02} \end{matrix} & \begin{matrix} T_x \\ T_y \\ T_z \end{matrix} \\
 \begin{matrix} \text{v1.y} \\ R_{10} \end{matrix} & \begin{matrix} \text{v2.y} \\ S_y R_{11} \end{matrix} & \begin{matrix} \text{v3.y} \\ R_{12} \end{matrix} & \\
 \begin{matrix} \text{v1.z} \\ R_{20} \end{matrix} & \begin{matrix} \text{v2.z} \\ R_{21} \end{matrix} & \begin{matrix} \text{v3.z} \\ S_z R_{22} \end{matrix} & \\
 0 & 0 & 0 & 1
 \end{bmatrix}$$

translation



Матрицы. Операции над матрицами

Когда мы выбираем объекты в 3D-редакторе, в инспекторе объектов справа вверху всегда видна локальная матрица трансформации (декомпозиция локальной матрицы). В ней *Position*, *Rotation* и *Scale* показываются относительно родителя. То есть родитель становится центром начала координат.

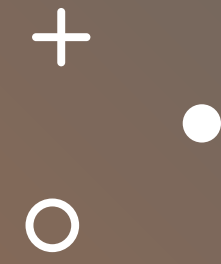
1. Родительско-дочерняя связь с точки зрения кода и матриц реализуется как:

$$\text{world_B} = \text{world_A} * \text{local_B}$$
$$\text{local_B} = \text{inverse}(\text{world_A}) * \text{world_B}$$

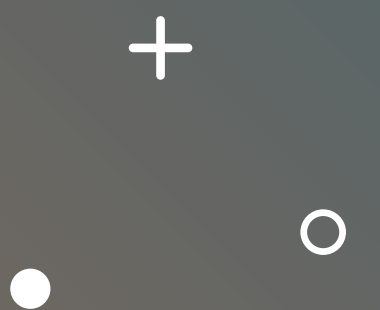
где *world_A* — глобальная (родительская) матрица трансформации, скажем, «героя».

local_b — локальная (дочерняя) матрица, к примеру, «пистолета».

world_b — это ее глобальная (мировая) матрица трансформации.



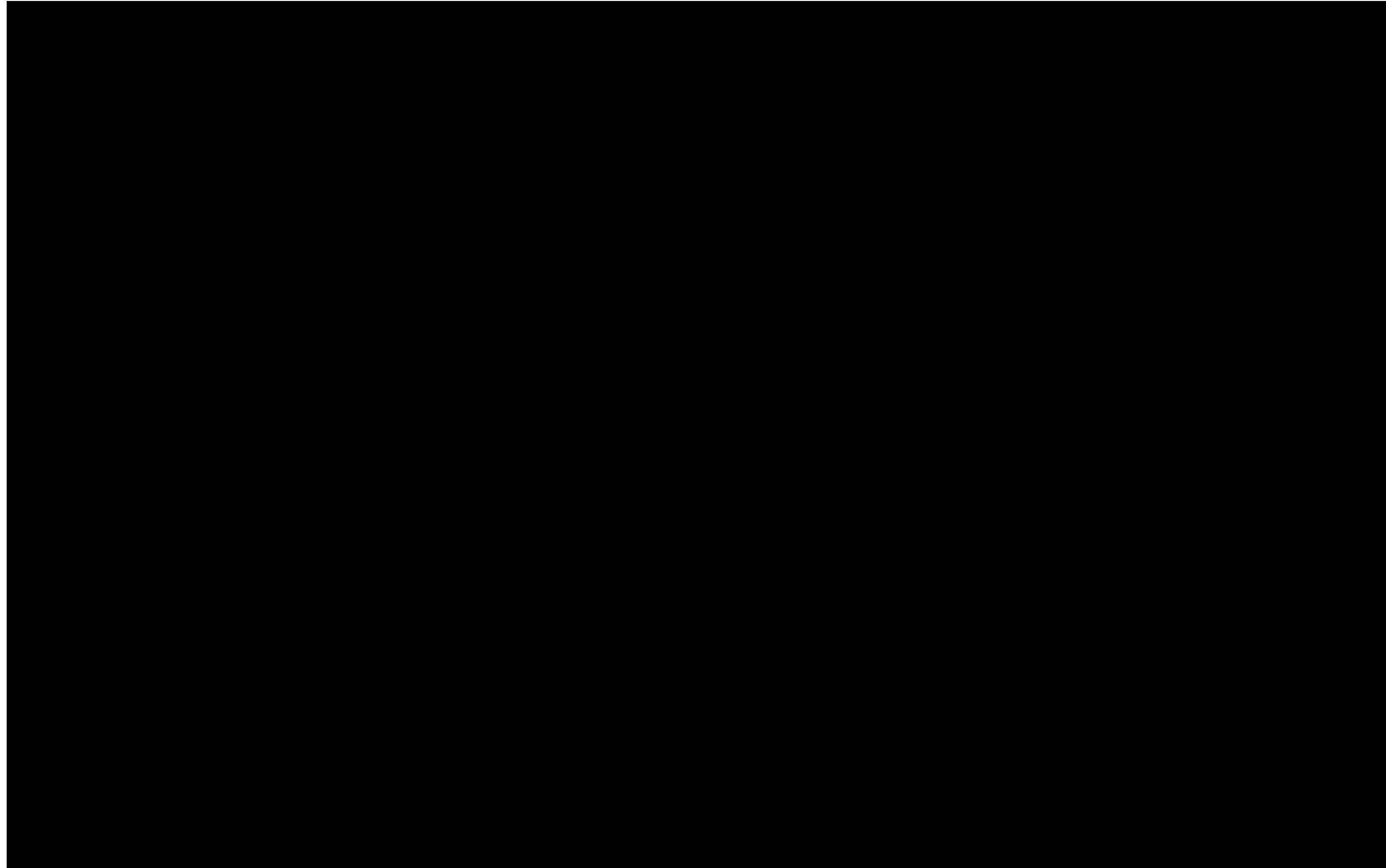
История компьютерной графики



Доисторическая эпоха

- История графики в кино началась почти сразу же, как появились первые компьютеры. Ещё в поздних 1940-х годах пионер компьютерной анимации Джон Уитни использовал вычислительные способности машин для создания примитивных узоров, которые затем переносились на плёнку. Он экспериментировал с методами механической анимации на девайсах, построенных на основе военной техники времён Второй мировой.
- А первым фильмом, в котором использовалась компьютерная графика, считается классическая картина Альфреда Хичкока «Головокружение» (1958). Именно Джон Уитни вместе с графическим дизайнером Солом Бассом создал вступительную сцену ленты.

Альфред Хичкок. Головокружение (1958)



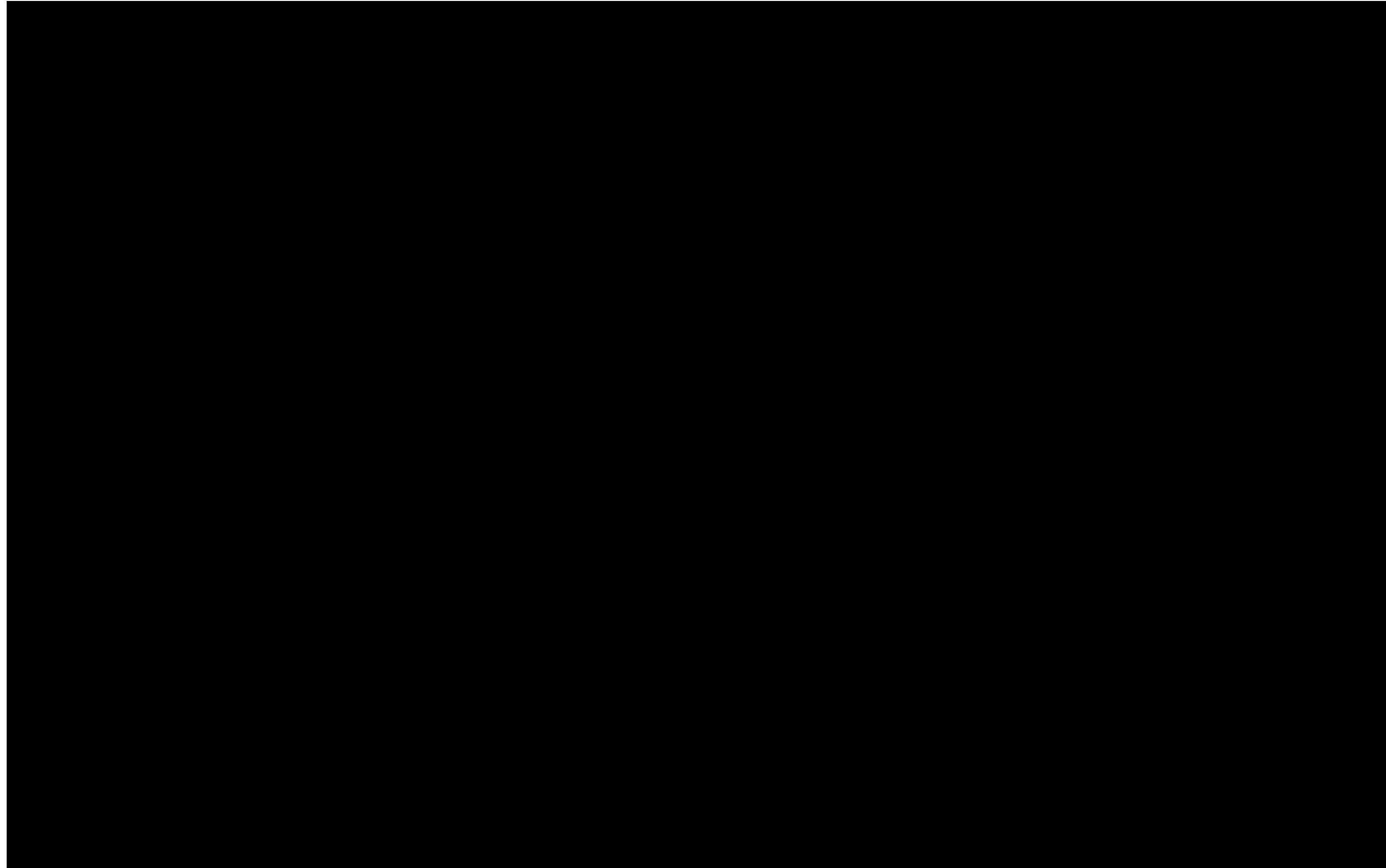
Альфред Хичкок. Головокружение (1958)

- Эта сцена у современников вызывала чувство головокружения, чего и добивались Хичкок и Сол Басс. Дизайнер взял за основу мотив лестницы из фильма, на которой у главного героя кружилась голова. Безостановочный спуск по бесконечной лестнице по задумке Сола вызывал чувство тревоги, поэтому ему потребовалась механически точная анимация из отдельных рисунков, чего вручную добиться невозможно.

Доисторическая эпоха

- Шестидесятые годы XX века [ознаменовались](#) появлением первой компьютерной анимации, созданием трёхмерной каркасной модели, первой симуляцией полёта самолёта, сценой с захватом движения (motion capture) и появлением на экране первого говорящего CGI-персонажа. Список на первый взгляд внушительный, хотя современного зрителя эти сцены, конечно, вряд ли впечатлят. Впрочем, давайте взглянем на пару примеров.
- В 1961 году Джон Уитни создал первый в истории короткометражный анимационный фильм «Каталог» (Catalog).
- Образы из видеоролика вдохновили Стэнли Кубрика, который применил некоторые визуальные эффекты из «Каталога» в фильме «[Космическая одиссея 2001](#)». Впрочем, невесомость в легендарной ленте создавалась с помощью практических эффектов.

Анимационный фильм «Каталог» (1961)







Перспектива

Определите, где будет
расположена линия
горизонта: на уровне глаз
зрителя, выше или ниже.



Линия горизонта высоко — показываем
объекты с высоты птичьего полета.



Линия горизонта на уровне глаз —
обычный вид как в жизни, показываем
картинку с точки зрения персонажа.



Линия горизонта низко — показываем
нависание, смотрим на объекты снизу.



SmirnovSchool

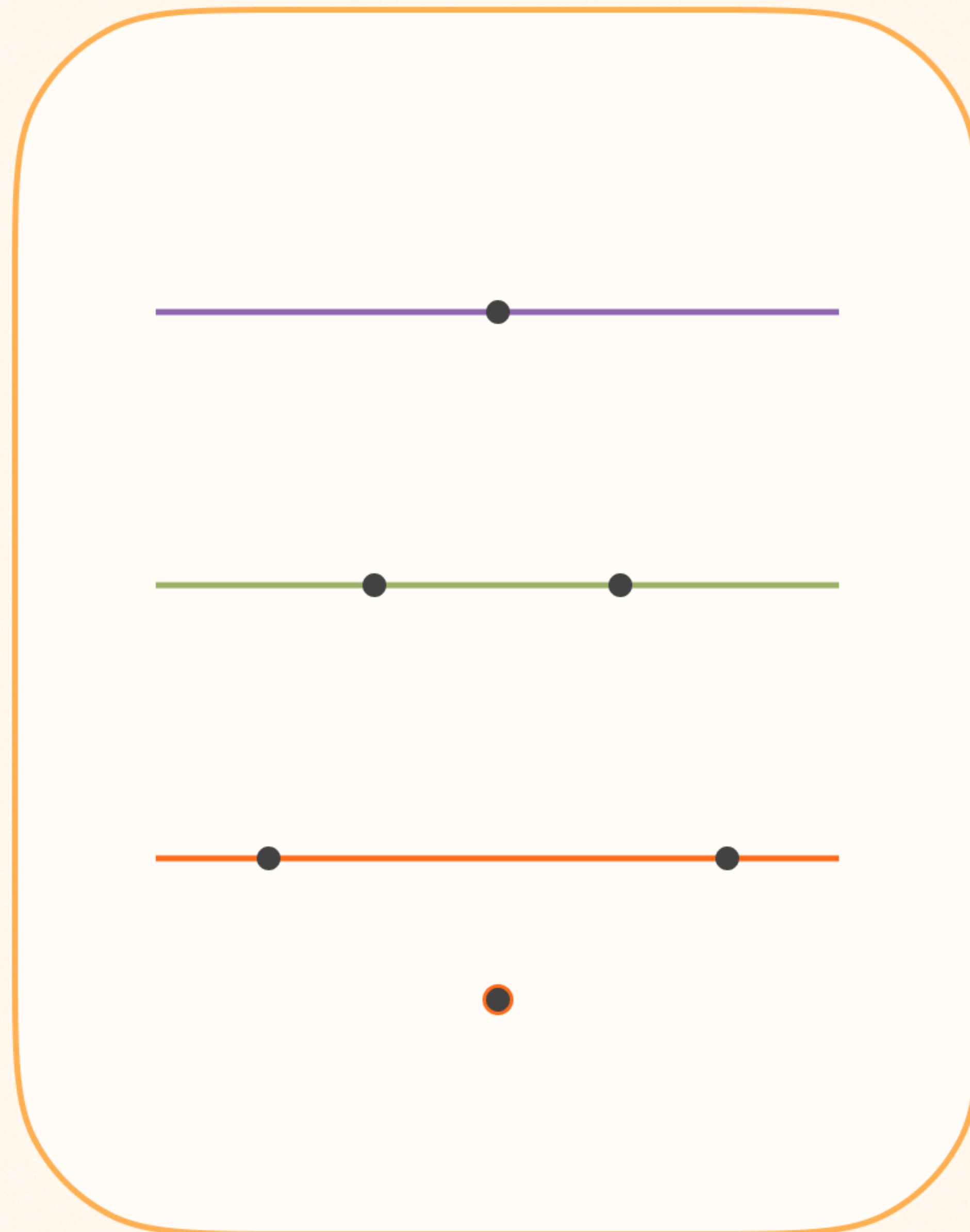
1

Как строить перспективу



Определите количество точек схода: 1, 2 или 3

-  1 - для спокойной статичной перспективы, симметрии.
-  2 - для угловой перспективы, самой распространенной и классической.
-  3 - для динамики, асимметрии, необычных ракурсов и масштабов.

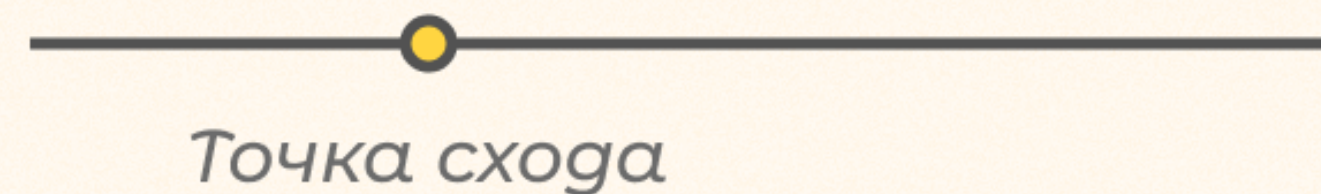


Перспектива с одной точкой схода

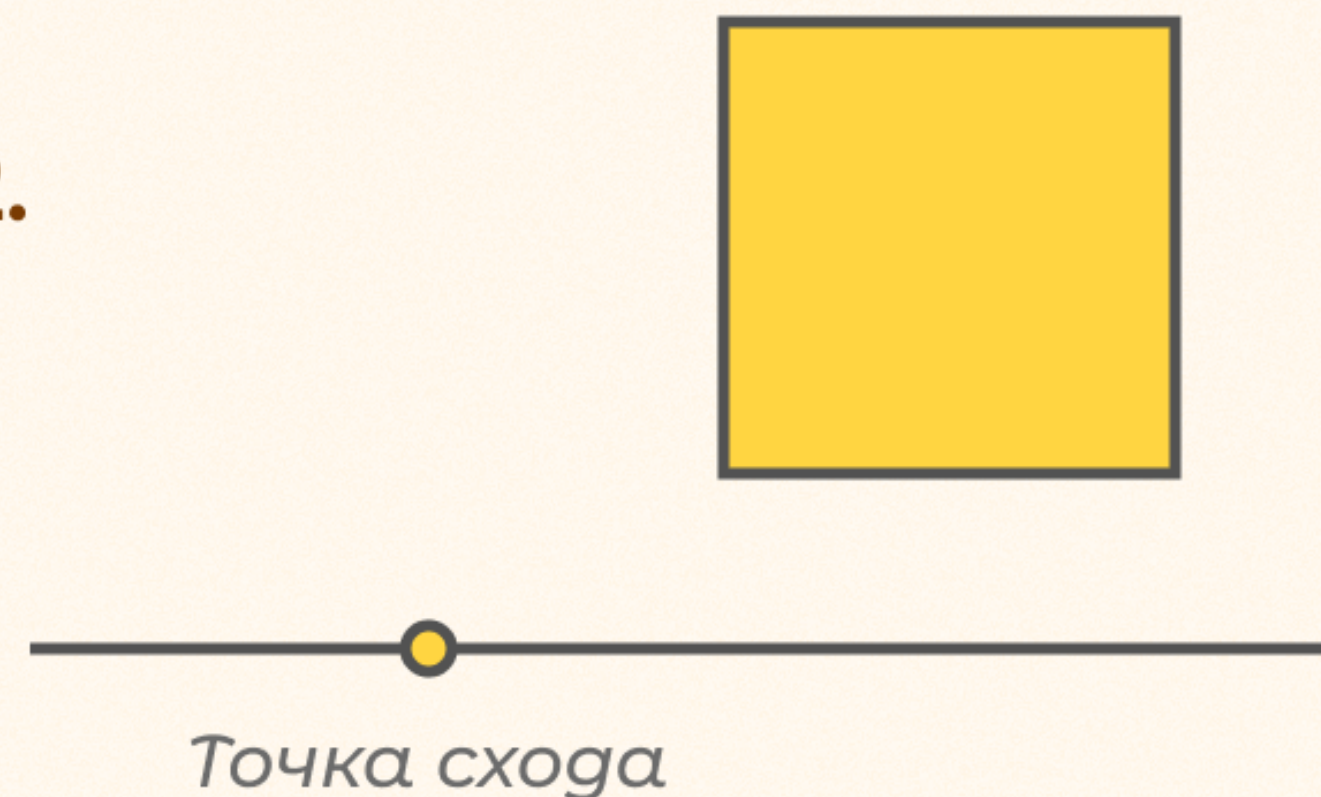
Вертикальные линии всегда перпендикулярны линии горизонта

1. Рисуем линию горизонта и ставим точку схода
2. Рисуем фронтальную сторону — она всегда будет без искажений
3. Из всех вершин проводим линии в точку схода — это ребра фигуры

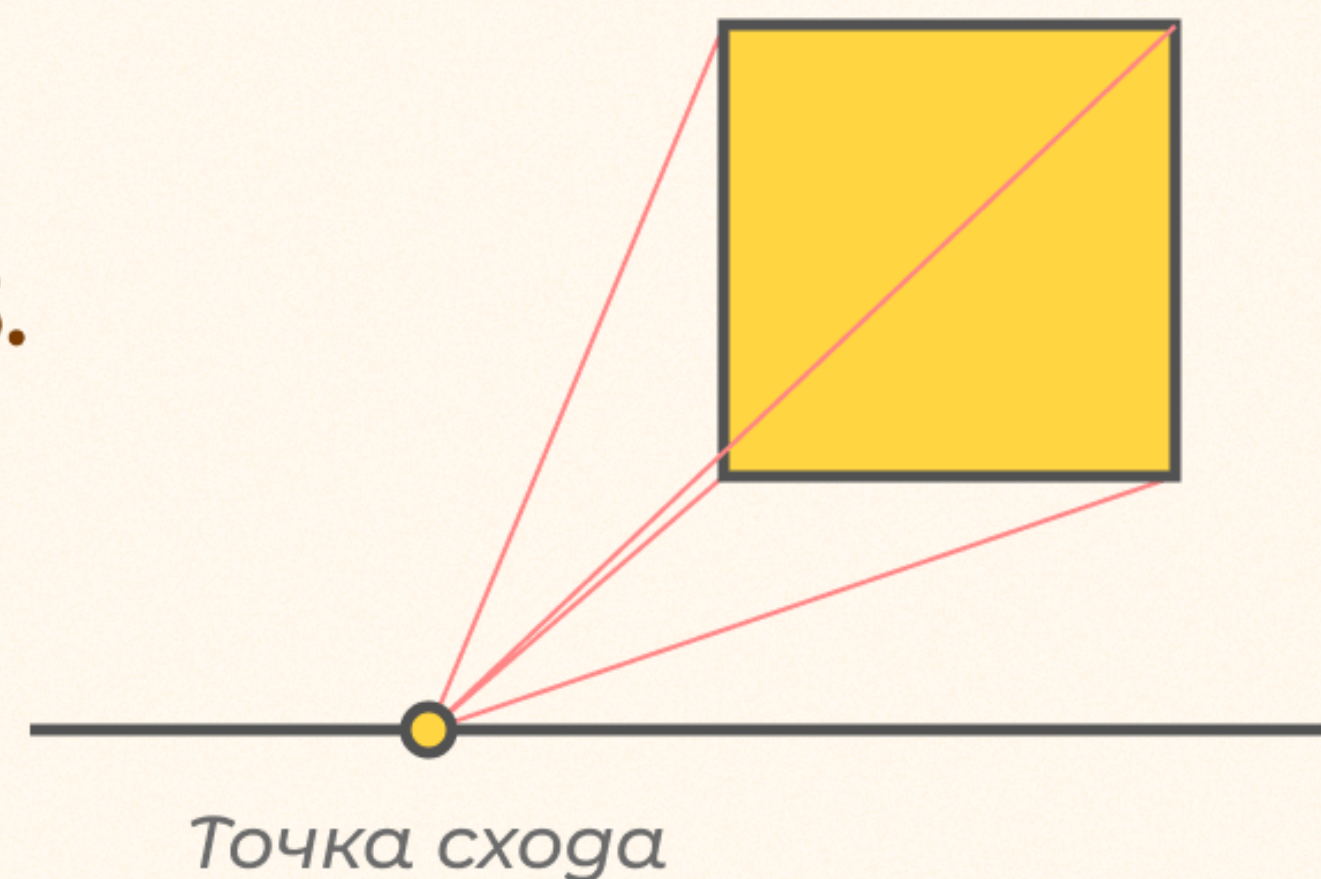
1.



2.



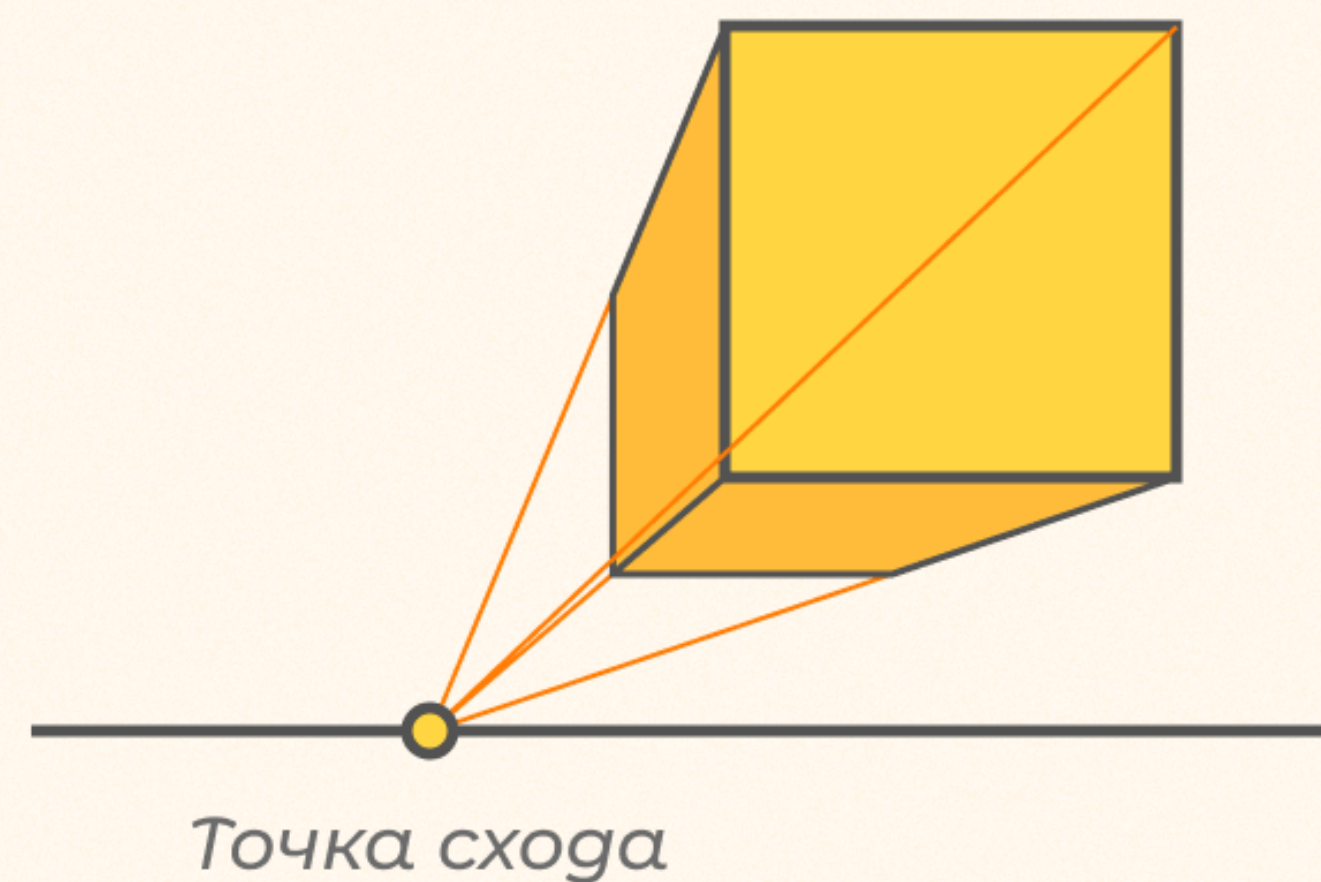
3.



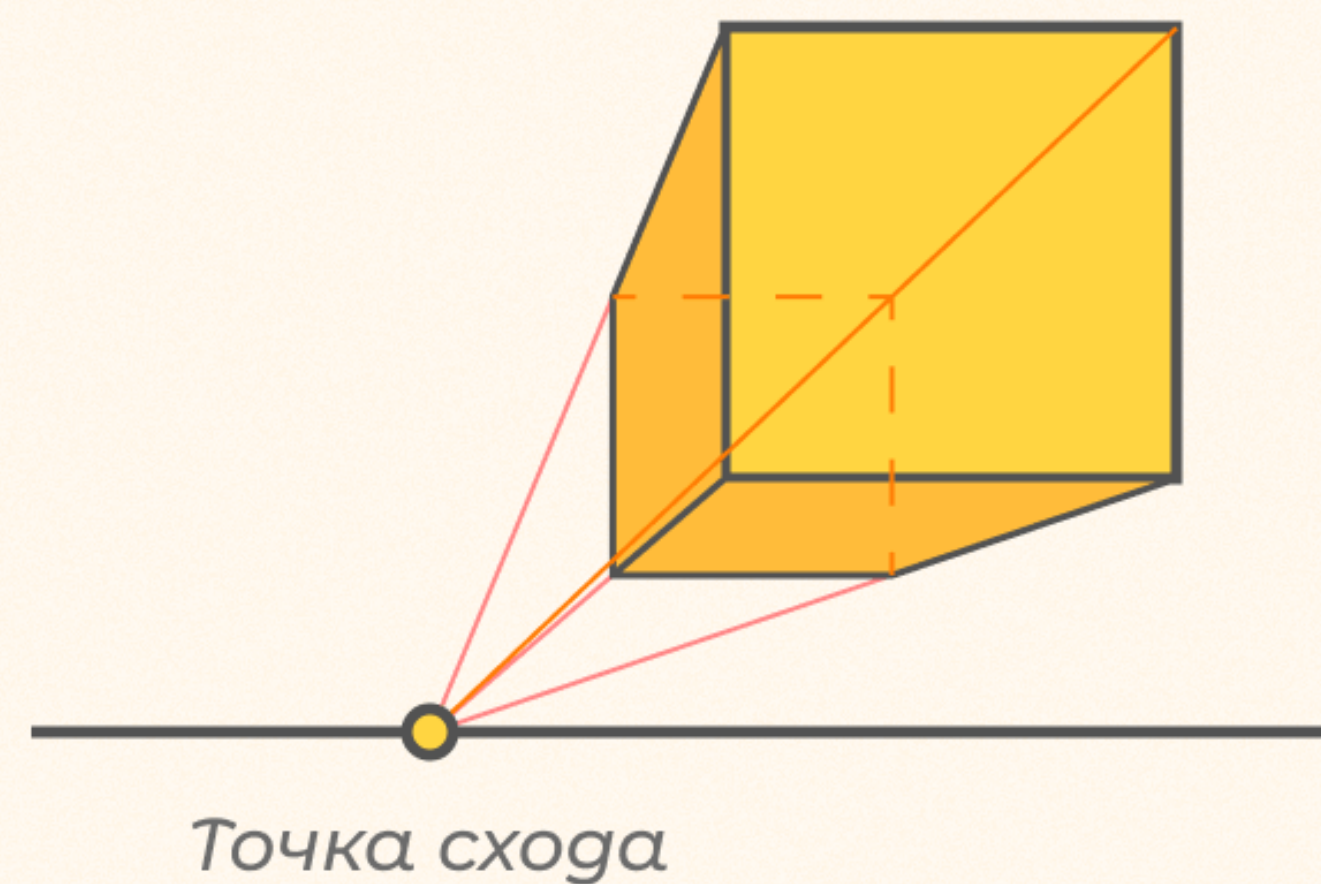
Перспектива с одной точкой схода

4. Рисуем дальние грани — проводим горизонтальные и вертикальные линии до пересечения с ребрами
5. Из получившихся углов проводим вертикаль и горизонталь до дальнего ребра, чтобы изобразить заднюю стенку.

4.



5.



Перспектива с двумя точками схода

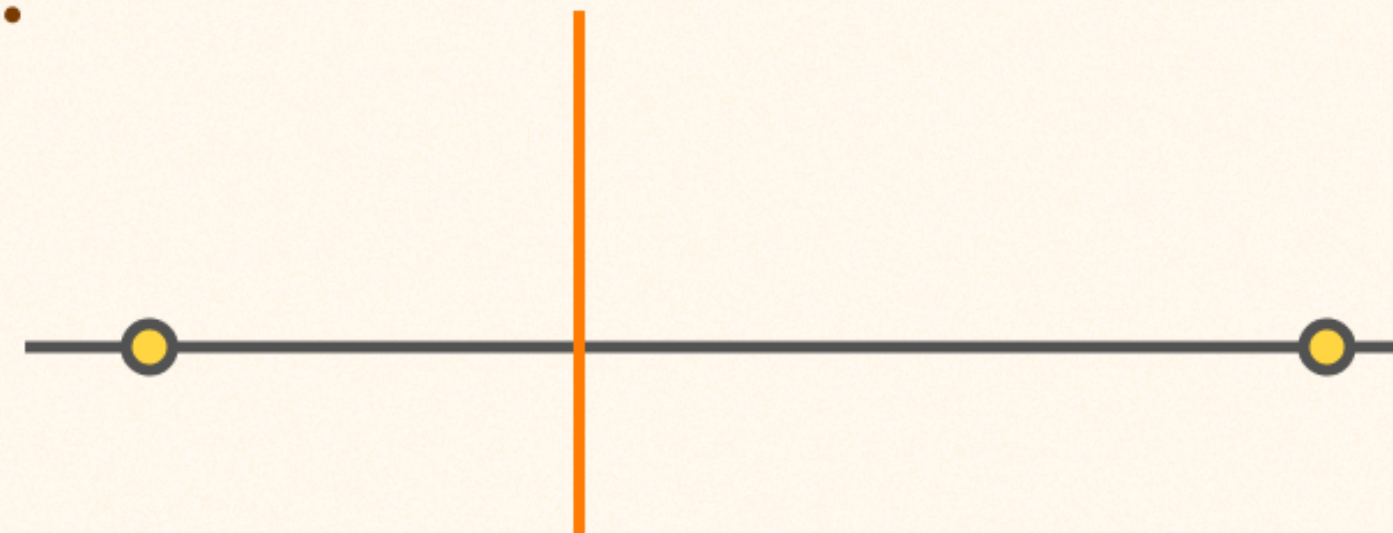
Вертикальные линии всегда перпендикулярны линии горизонта

1. Рисуем линию горизонта и ставим две точки схода
2. Рисуем вертикаль — ближайшее к нам ребро
3. От вершин ребра проводим линии в точки схода

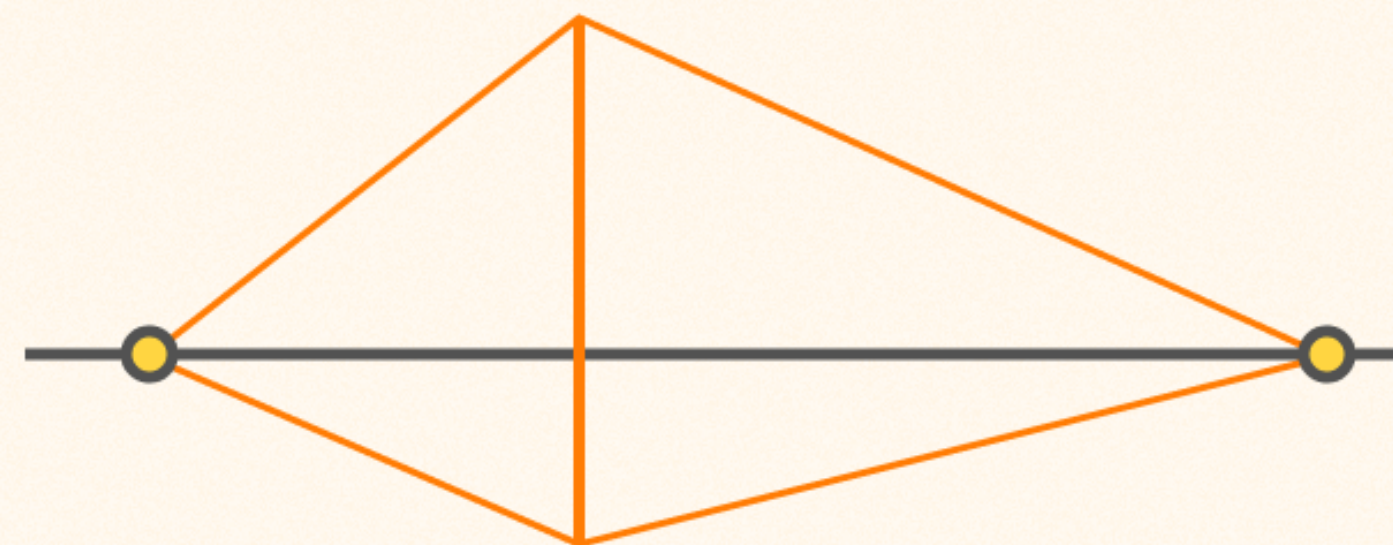
1.



2.



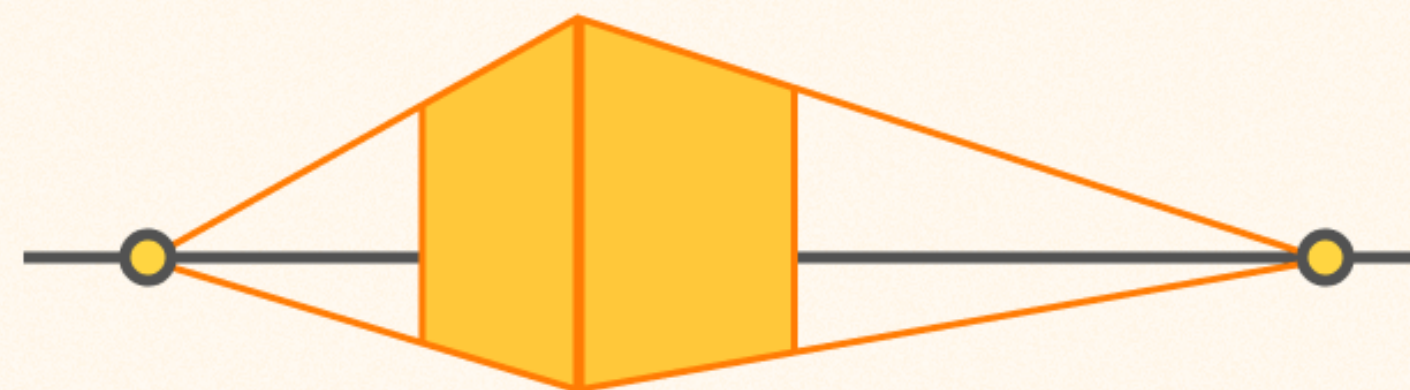
3.



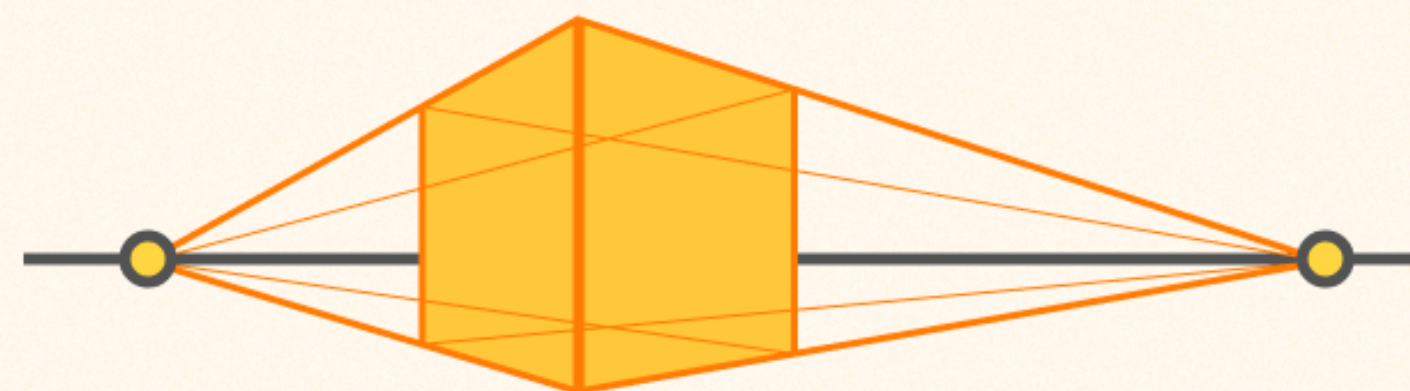
Перспектива с двумя точками схода

4. Проводим вертикали еще двух ребер, обозначая стороны куба
5. Из получившихся углов проводим линии в точки схода
6. Соединяем вертикалью дальние углы — получаем дальнее ребро.

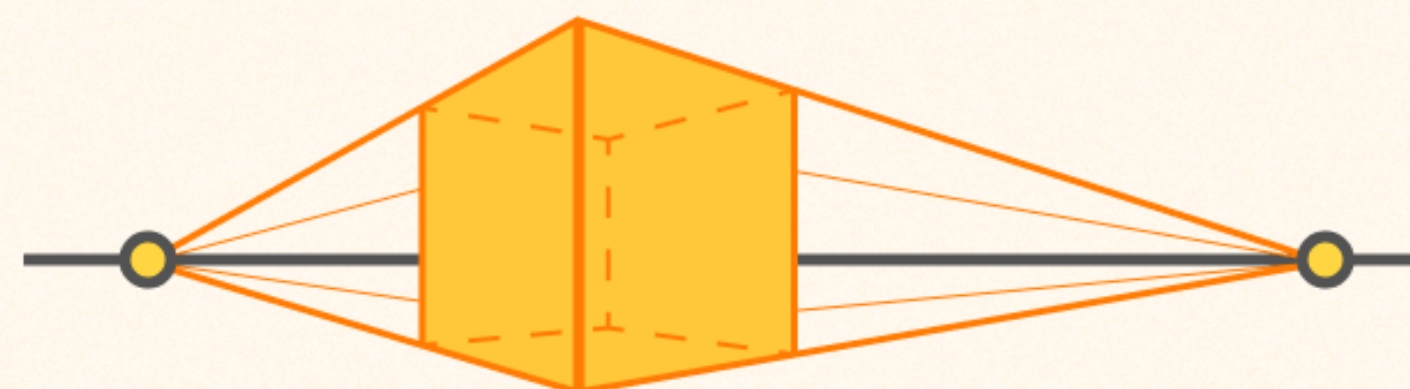
4.



5.



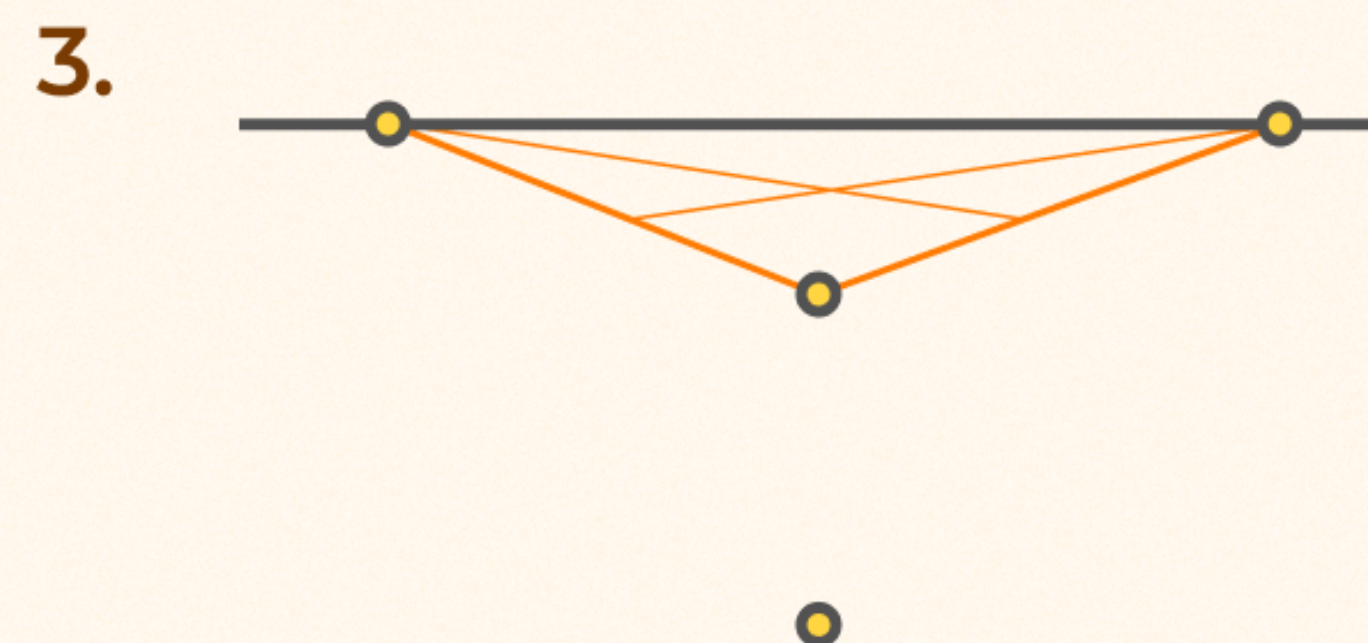
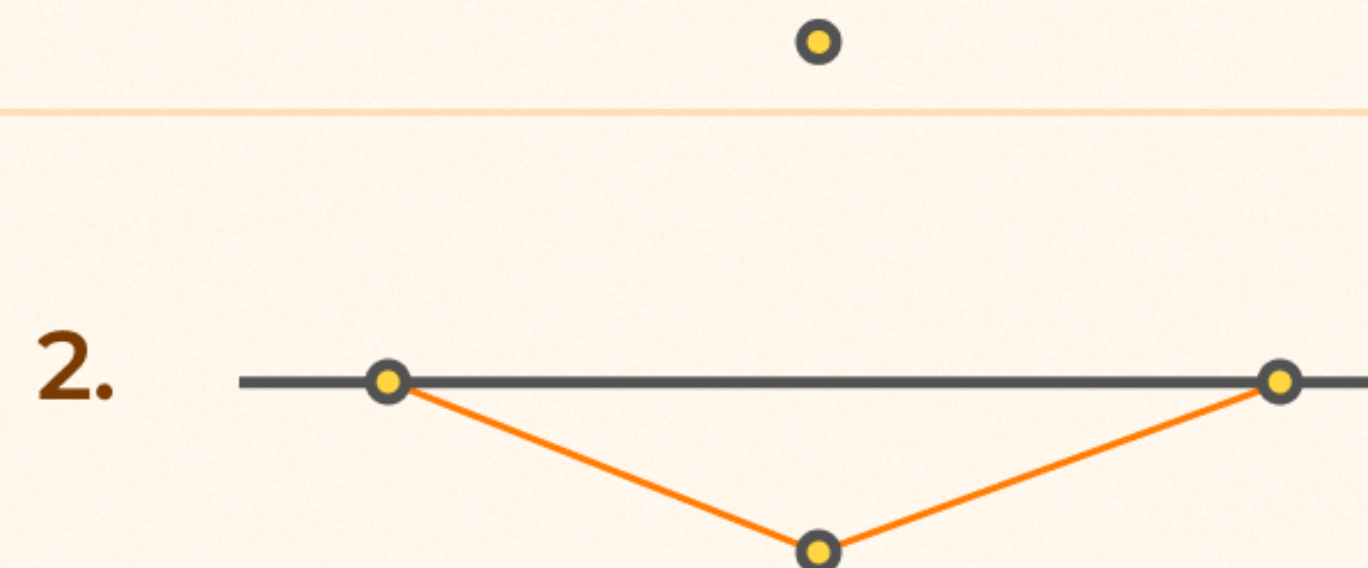
6.



Перспектива с тремя точками схода

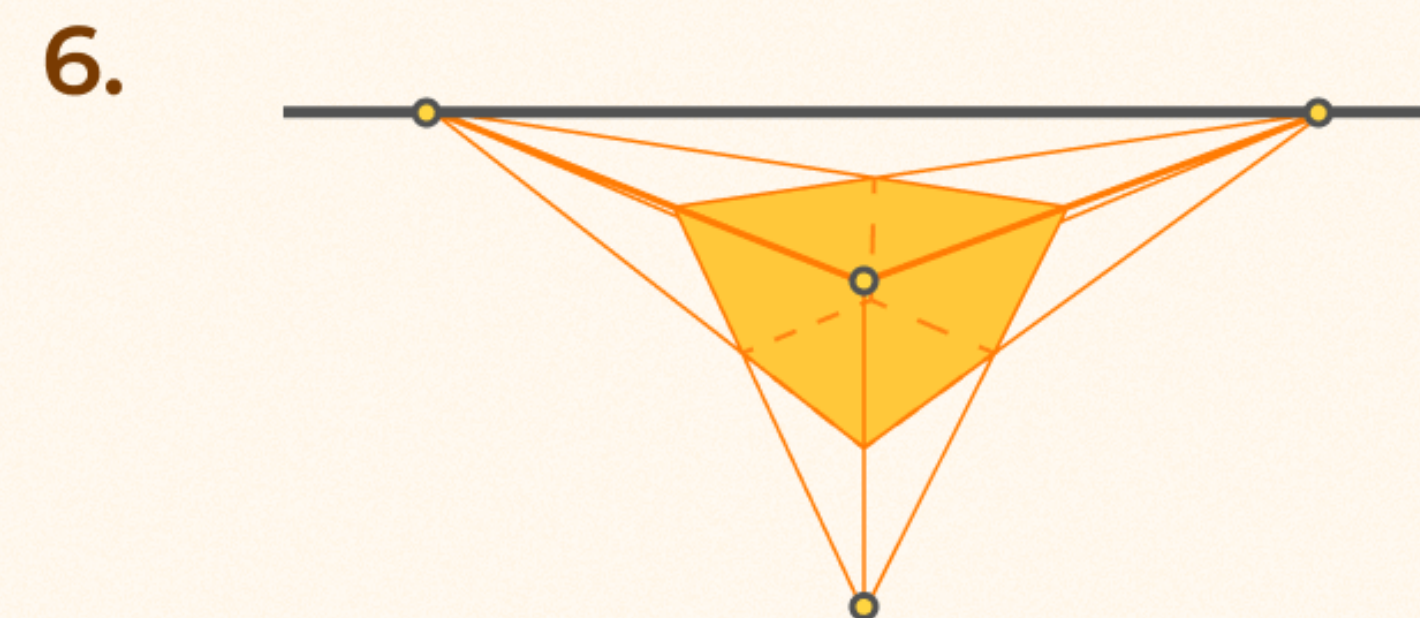
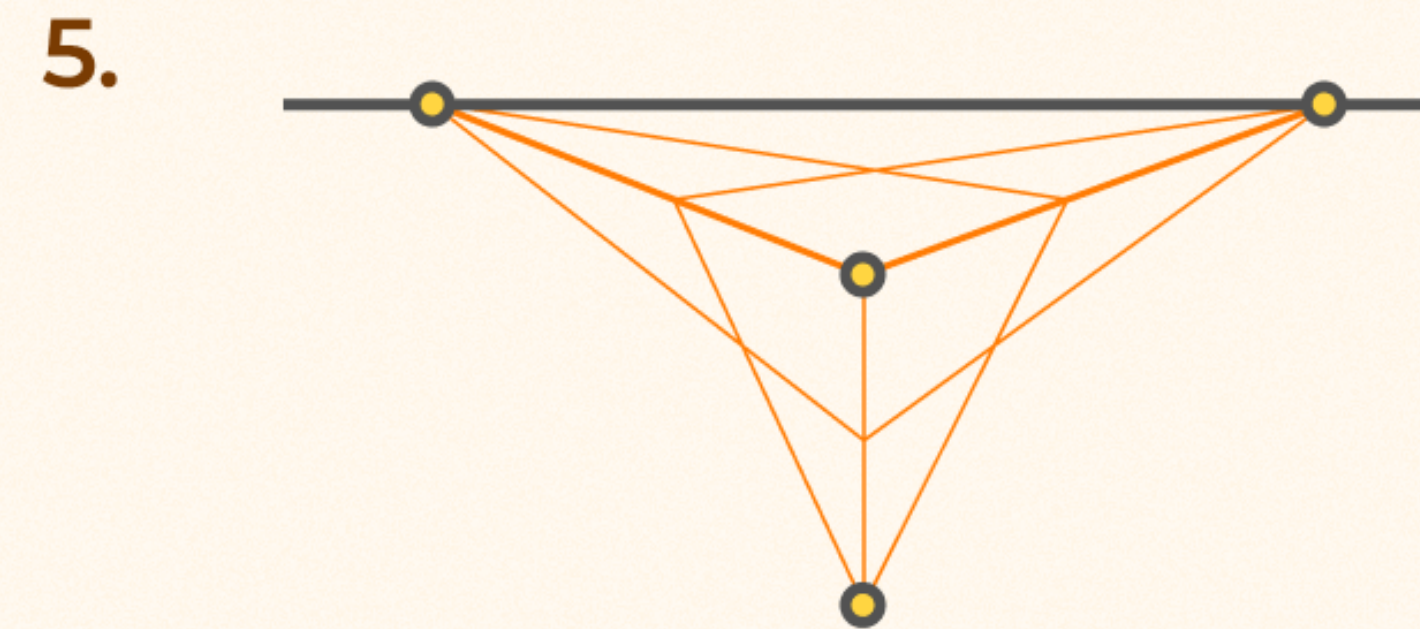
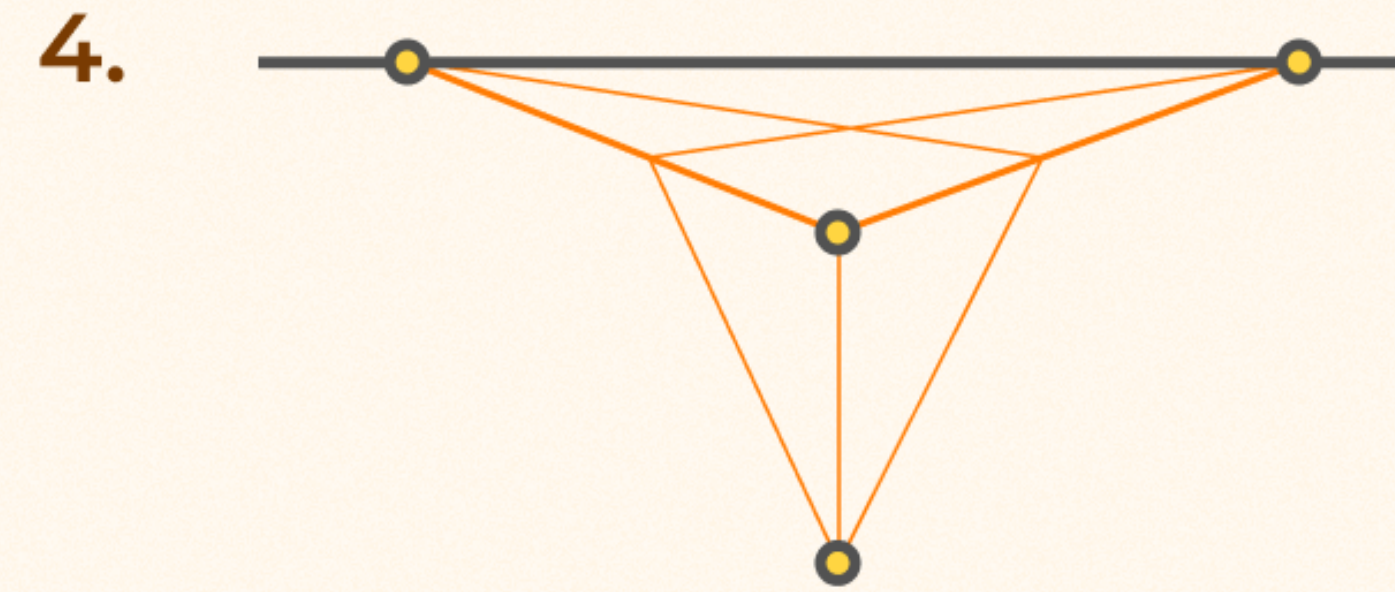
Вертикальные линии искажаются, как и горизонтальные

1. Рисуем линию горизонта. Ставим три точки схода: две на ЛГ и одна вне ее — к ней будут стремиться вертикали
2. Ставим точку для ближайшего к нам угла фигуры и проводим из нее линии к точкам схода на ЛГ
3. Проводим в точки схода еще две линии — получим верхнюю грань куба



Перспектива с тремя точками схода

4. Из получившихся углов проводим вертикали в третью точку схода
5. Из точек схода проводим линии, которые определяют нижние ребра фигуры
6. Из получившихся углов проводим линии в точки схода на горизонте, чтобы обозначить «невидимые» грани. Не забываем про самую дальнюю грань.



ЭТО ИЗОМЕТРИЧЕСКАЯ
ПЕРСПЕКТИВА



ЭТО ЛИНЕЙНАЯ
ПЕРСПЕКТИВА



ЭТО ВОЗДУШНАЯ
ПЕРСПЕКТИВА



ЭТО МОЯ ПЕРСПЕКТИВА
НАУЧИТЬСЯ НОРМАЛЬНО
РИСОВАТЬ



Алгоритм построения окружности методом
средней точки

Рисование окружностей на растровом дисплее

Аналог Брезенхема для кривых второго
порядка

Аналогия с алгоритмом Брезенхема

Брезенхем (линия)

Мидпоинт (окружность)

Движение вдоль X

Движение вдоль X (1-й октант)

Ошибка от прямой

Ошибка от окружности

Выбор: E или NE

Выбор: E или SE

Инкремент ошибки

Инкремент ошибки + зависит от позиции

Где применяется?

- Построение окружностей, арок, скруглений в GUI
- Векторные шрифты (буквы с дугами)
- Игровая графика (зоны поражения, индикаторы)
- ЧПУ-станки (дуги интерполяции)
- CAD-системы (базовые примитивы)
- Лежит в основе библиотек типа Cairo, Skia, а также аппаратных растризаторов

Математика

- Уравнение окружности:
 $x^2 + y^2 = R^2$
- Функция решения:
 $F(x, y) = x^2 + y^2 - R^2$
- $F = 0$ — на окружности
- $F > 0$ — снаружи
- $F < 0$ — внутри
- Симметрия: строим $\frac{1}{8}$ дуги от $x=0$ до $x=y$, остальное — отражениями.
- Берём среднюю точку между двумя кандидатами (E и SE). Подставляем её координаты в F. Знак говорит, где находится окружность. Это и есть «мидпойнт» — главное отличие от Брезенхема, который рассматривал только расстояние вертикали.

Псевдокод алгоритма (1-й октант)

```
function drawCircle(R):  
    x = 0  
    y = R  
    d = 1 - R      // начальное значение ошибки (midpoint)  
  
    while x <= y:  
        plot(x, y)  // и 7 симметричных точек  
  
        if d < 0:  
            // выбрать E (восток)  
            d = d + 2*x + 3  
        else:  
            // выбрать SE (юго-восток)  
            d = d + 2*(x - y) + 5  
            y = y - 1  
  
        x = x + 1
```

Алгоритм инициализирует $d = 1 - R$ — это не расстояние, а значение функции в средней точке между E и SE (при $x=0, y=R$, середина в $(1, R-0.5)$). Обратите внимание, что y уменьшается только при $d \geq 0$. Все операции целочисленные, кроме одного вычитания.

Разбор примера

x	y	d	Действие	Пиксели (x,y)
0	5	$1-5 = -4$	E	(0,5)
1	5	$-4+2*1+3=1$	SE ($d \geq 0$)	(1,5), y=4
2	4	$1+2*(1-5)+5 = -2$	E	(2,4)
3	4	$-2+2*3+3=7$	SE	(3,4), y=3
4	3	$7+2*(3-4)+5=5?$ (ошибка)	SE	(4,3), y=2

Симметрия окружности (все 8 октантов)

- Вызов `plot(x,y)` 8 точек:
- $(x, y), (y, x), (-x, y), (-y, x),$
 $(x, -y), (y, -x), (-x, -y), (-y, -x)$
- Благодаря симметрии мы строим лишь 1/8 окружности. Остальное заполняем отражениями. Это даёт колоссальную экономию — примерно в 8 раз быстрее, чем полный перебор.

Задание

- $R=4$. Повторите 5-6 шагов алгоритма построения окружности методом средней точки.
- Даны:
 $x=0, y=4, d=1-4 = -3$

Ответ

Шаг	x	y	d	выбор	нов. d
1	0	4	-3	E	$-3+2*0+3 = 0$
2	1	4	0	SE	$0+2*(1-4)+5 = -1, y=3$
3	2	3	-1	E	$-1+2*2+3 = 6$
...	...	...	...	...	...

Обобщаем

- Алгоритм построения окружности методом средней точки — обобщение Брезенхема на кривые 2-го порядка
- Использует знак функции в средней точке
- Целочисленный, без деления
- Строит 1/8, остальное — симметрия